ChatGPT

Long-Term Architecture and Agent Workflow Plan (2026-06-28)

1 Purpose and Context

The initiative tracker project has crossed the point where proving web/headless feasibility is no longer the central campaign. The **DM browser** and **headless host** milestones (A/B) are marked *complete enough* in the master tracker — DM workflows are viable and `serve_headless.py` with `HeadlessRoot` provides a real non-window host mode [1]. The team is now focused on making sessions reliable, untangling desktop-owned logic from backend authority, and preparing for a future server-resident runtime [2]. At the same time the **AGENTS.md** guidance stresses that repository files are durable and chat is volatile; anything important must be written back into the repo [3]. Deep research, speculative design, and decisions need to be converted into explicit, repository-stored documents that future agents can follow.

This plan summarises recent documentation, identifies patterns that work and those that do not, and outlines concrete steps for extracting the web server, reducing latency, and formalising agent routines. All recommendations are grounded in the latest repository docs and ignore stale plans unless they are expressly referenced [4].

2 Observed Patterns and Recent Evidence

2.1 Milestones and direction

The `majorTODO.md` snapshot shows that DM browser viability and Tk-optional host paths are considered complete; the backend/service seam work is in progress; and exploration of a server-resident runtime is explicitly labelled *Exploration only* [5]. The document warns against assuming that the current runtime shape is the end state just because headless/browser viability exists [6]. Current work should therefore focus on stabilisation and corrective passes rather than new milestone chasing.

2.2 Current work ledger and refusal rule

The `docs/work_items/current_work.md` ledger acts as the authoritative source for active tasks. It instructs the orchestrator not to invent work items from old docs or historical runtime reports when none are listed [7]. Agents must therefore consult this ledger first and request developer input if no current item is active.

2.3 Service seam and concurrency model

The **DM Web Migration** document details the existing backend/service seam:

- The `CombatService` owns combat/session state and exposes HTTP endpoints such as `GET /api/dm/combat`, `POST /api/dm/combat/start`, HP adjustments and condition changes [8].
- HTTP route handlers run on the FastAPI thread and call tracker methods directly [9].
- `CombatService` holds a re-entrant `threading.RLock` to serialise concurrent mutations from both web and desktop callers, and broadcasts updates via WebSockets [10].

This design proves that a backend authority can serve multiple clients (desktop and web) but still embeds synchronous tracker calls in the request thread. Heavy operations (e.g., building tactical snapshots) still block request handling, causing latency and stalling the web server.

2.4 Host modes

There are two host modes:

1. **Desktop/Tk host** – start the desktop app, enable the LAN server from the menu, and access `/dm` or `/dm/map` from other devices [11].
2. **Headless/server host** – run `python3 serve_headless.py` with `INIT_TRACKER_HEADLESS=1`; this swaps `tk.Tk` for `HeadlessRoot` and avoids opening a GUI [12].

Headless mode demonstrates that the runtime can live without a Tk mainloop, but it still runs the same `InitiativeTracker` and directly serves FastAPI routes [13].

2.5 Tactical map and latency issues

A runtime report from May 2026 explains that disabling the tactical map by default improved combat-loop latency but broke the dedicated DM map/control workspaces. The hotfix added **workspace-aware routing**: combat-lite endpoints remain lightweight, while `/dm/map` and `/dmcontrol` always request tactical payloads and annotate requests with `?workspace=dmcontrol` [14]. This shows that conditional payloads and context-aware snapshot building can reduce latency for common operations while still serving heavy maps when needed.

2.6 Disparity between player and DM control surfaces

The monster-control plan notes that the player LAN page is a robust combat surface with modal spell casting, movement cycling and reactions, backed by `PlayerCommandService` and `player_command_contracts.py` [15]. In contrast, the DM console offers only a free-text monster action input and manual damage entry [16]. This disparity means DM workflows involve more manual work and delay; a future DM control interface should approach feature parity with the player surface.

2.7 Agent discipline

The **AGENTS.md** guidelines emphasise that the repository is the durable record and chat history is scratch space [3]. Agents are instructed to read only named files, prefer `docs/work_items/current_work.md`,

and avoid inspecting `majorTODO.md` or historical reports unless explicitly named [4]. This ensures that work decisions are grounded in current context and not stale plans.

3 Decisions and Recommendations

3.1 Extract the web server from the runtime

Problem: At present the FastAPI web server is started from `serve_headless.py` and `dnd_initative_tracker.py`, which also instantiate the `InitiativeTracker`. HTTP route handlers call tracker methods directly, so expensive mutations (loading a monster pack, building tactical snapshots, loading YAML) block the request thread [17]. The runtime report shows that gating heavy map builds behind an environment flag improved latency [14], but this is a workaround, not a final solution.

Decision: Introduce a clear **server–runtime boundary**. The FastAPI app should start first and treat the runtime as a service object. All HTTP/WebSocket calls should be thin adapters that submit commands or read cached snapshots.

- **Runtime service:** Wrap the existing `InitiativeTracker` inside a runtime service that exposes methods for commands (e.g., `start_combat`, `advance_turn`, `apply_damage`) and read-only snapshot access. The service maintains a snapshot cache and a command queue. Heavy operations run in worker threads or async tasks, leaving the HTTP event loop free.
- **Command queue:** Replace direct tracker calls in route handlers with commands enqueued to a background worker. This isolates latency spikes and allows bulk operations (e.g., loading a monster pack) to complete without blocking the server.
- **Cached snapshots:** After each mutation, the runtime invalidates and rebuilds the necessary snapshot(s). Read routes (`GET /api/dm/combat`) return the latest cached snapshot, not a freshly built map unless a workspace requires it. The workspace context mechanism from the May 2026 hotfix should be preserved: only `/dm/map` or `/dmcontrol` requests with appropriate query parameters cause a tactical map rebuild [14].
- **Thread safety:** Retain the `threading.RLock` for mutation operations inside the runtime service to preserve consistent state [10].

3.2 Re-organise host entry points

The two host modes should remain but be cleaned up:

- `serve_headless.py` becomes a thin launcher that sets environment variables and creates the FastAPI app through an **application factory**. It should not import or instantiate the tracker directly; instead it should create the runtime service and pass it into the app factory.
- `dnd_initative_tracker.py` should eventually be deprecated or simplified. For backward compatibility, it can still host the Tk UI and start the LAN server, but the runtime logic should reside in the same runtime service used by headless mode.
- Provide a unified `app.py` module (e.g., `init_tracker_server/app.py`) that constructs the FastAPI app, registers routes, websockets and middlewares, and takes a runtime service instance as a dependency.

3.3 Improve DM control parity

The disparity between player and DM surfaces is a known pain point [18]. While not the immediate focus of server extraction, the long-term architecture should include:

- A robust monster-control interface with structured actions, spells and templates instead of free-text fields.
- A normalised monster capability schema backed by YAML or JSON to allow automated resolution of attacks and spells.
- Expanded `CombatService` or a separate `MonsterCommandService` to handle DM commands similar to `PlayerCommandService`.

These plans should be documented separately (see `docs/dm-monster-control-surface.md`) and prioritised after the server extraction.

3.4 Documentation practices and agent routine

- **Use the current work ledger:** Always consult `docs/work_items/current_work.md` for the active work item. If none exists, the agent must stop and ask the developer for direction instead of inventing tasks [7].
- **Write durable documents:** Follow AGENTS.md guidance: treat repository files as the durable record and conversations as scratch space [3]. Each significant design decision or migration should result in a new Markdown file under `docs/architecture/` or `docs/work_items/` with a descriptive name and date. For example, this plan could live in `docs/architecture/long_term_server_extraction_20260628.md`.
- **Prefer recent docs:** When researching or planning, prioritise `current_work.md` and the active task packet over historical documents; avoid using `majorTODO.md` or old runtime reports unless named [4].
- **Keep decisions tied to evidence:** Include citations from code or docs for each assumption. Use the pattern seen in this document (inline citations pointing to specific lines) to aid future agents.
- **Update the ledger:** When a decision becomes an active work item, add it to the `Active Work Table` in `current_work.md`. When completed, summarise the evidence and link to the decision document.

3.5 Future exploration

The master tracker notes an eventual TypeScript-first, server-resident runtime [19]. This is exploratory and should not be treated as active. The immediate focus is stabilisation and extraction. Once the runtime has been properly decoupled and performance issues addressed, the team can revisit the language choice and potentially reimplement the runtime in TypeScript or another language.

4 Agent Workflow for Future Tasks

Agents working in this repository should follow this routine:

1. **Sync and read:** Fetch the latest `docs/work_items/current_work.md` and any file named in the active work item. Do not read the entire repository; only access named files.

2. **Validate context:** If no work item is active, stop and ask the developer whether to open a new bug report, start a planning pass, continue to the next recovery gate, perform smoke testing, commit changes or deploy [20].
3. **Perform bounded tasks:** Execute exactly the work specified. Avoid opportunistic changes or broad refactors outside of the described scope [21].
4. **Document findings:** For any research, design or decision, create a Markdown file under an appropriate `docs/` subfolder (e.g., `docs/runtime_reports`, `docs/architecture`, `docs/work_items/active`). Summarise confirmed facts separately from assumptions, include references to relevant code/docs, and outline next steps.
5. **Update work tables:** When a decision becomes an active work item, append a row to the `Active Work Table`. When a task is completed, add an entry to the `Recently Completed Table` with a brief note and link to the evidence [22].
6. **Ask for review:** Before committing changes that affect code or production, request developer review. Do not deploy or restart services without explicit approval [23].
7. **Maintain citation discipline:** Always cite lines from docs or code that support each decision. Avoid unsourced assumptions.

5 Next Steps

Based on the decisions above, the recommended long-term work items are:

1. **Draft a server-extraction design document.** Create `docs/architecture/server_extraction_plan_20260628.md` describing the runtime service class, command queue, snapshot cache, and app factory. Include diagrams and pseudo-code.
2. **Refactor `serve_headless.py` and `dnd_initative_tracker.py`.** Introduce `init_tracker_server/app.py` as the canonical FastAPI app factory. Update both entry points to use the runtime service instead of instantiating `InitiativeTracker` directly.
3. **Implement command queue and snapshot caching.** Build a minimal thread or asyncio queue for mutations; return cached snapshots in GET routes. Preserve the RLock for thread safety.
4. **Update route handlers and websocket broadcasts.** Modify routes to enqueue commands and check workspace query parameters when deciding whether to build tactical maps [14].
5. **Enhance DM control surfaces.** Prioritise parity with the player command interface by developing a structured monster control API and UI. Use the ideas in `dm-monster-control-surface.md` and schedule this after server extraction.
6. **Document and track each step.** For each significant change, create a corresponding doc file and update `current_work.md` accordingly. This ensures that all agents and developers share the same understanding and can trace decisions back to evidence.

---

[1] [2] [5] [6] [19] raw.githubusercontent.com
https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/majorTODO.md

[3] [4] [21] [23] raw.githubusercontent.com
https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/AGENTS.md

[7] [20] [22] raw.githubusercontent.com
https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/docs/work_items/current_work.md

8 9 10 11 12 13 17 raw.githubusercontent.com

https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/docs/dm-web-migration.md

14 raw.githubusercontent.com

https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/docs/runtime_reports/hotfix_dm_map_combat_lite_regression_20260523.md

15 16 18 raw.githubusercontent.com

https://raw.githubusercontent.com/jeeves-jeevesenson/init-tracker/main/docs/dm-monster-control-surface.md